# A Self-Attentive Meta-Optimizer with Group-Adaptive Learning Rates and Weight Decay

JiangBo Zhao ZhaoXin Liu


## Abstract

Adaptive optimizers like AdamW apply uniform hyperparameters across all parameter groups, ignoring heterogeneous optimization dynamics across layers and modules. We address this limitation by proposing **MetaAdamW** – a new optimizer that integrates a self-attention mechanism to dynamically modulate per-group learning rates and weight decay. The modulation factors are produced by a lightweight Transformer encoder that operates on statistical features (gradient norms, momentum norms, correlations) extracted from each parameter group. To train the attention module, we introduce a meta-learning objective that combines gradient alignment, loss decrease, and generalization gap. A key novel contribution is the extension of homoscedastic uncertainty weighting [3] (HUW) with task-specific priorities that directly scale the regularization terms – enabling domain knowledge to guide automatic loss balancing. Extensive experiments on five diverse tasks—time series forecasting (ETT), language modeling (WikiText-2), machine translation (Multi30k), image classification (CIFAR-10), and sentiment analysis (IMDB) — demonstrate that MetaAdamW consistently outperforms the standard AdamW baseline in terms of validation loss, accuracy, or perplexity. Depending on the task, MetaAdamW either reduces overall training time (by up to 17.11%) or improves performance (by up to 11.08%) while introducing only moderate overhead; in some cases, it can also mitigate issues of insufficient convergence caused by premature early stopping. Ablation studies validate the effectiveness of each component, including feature versions, grouping strategies, and the proposed priority-injected uncertainty weighting.


**Keywords:** AdamW, Self-Attention, Meta-Learning, Group-Aware Optimization, Homoscedastic Uncertainty Weighting.

## 1 Introduction

Effective optimization algorithms are critical for deep learning success. Adam [4] and its variant AdamW [6] have become the de facto standard because of their adaptive learning rates and decoupled weight decay. However, a fundamental limitation remains: they apply identical hyperparameter settings (learning rate, weight decay) to all parameters irrespective of layer type or module function. In practice, embeddings, attention heads, and feed-forward networks exhibit markedly different optimization dynamics. This uniformity can lead to suboptimal convergence and impaired generalization – a problem we directly target in this work.

To overcome this limitation, we introduce MetaAdamW – a principled extension of AdamW that incorporates a self-attention mechanism for dynamic, per-group modulation of learning rates and weight decay. Our key idea is to treat each parameter group (semantically coherent sets of parameters) as a token, extract gradient and momentum statistics as features, and apply a lightweight Transformer encoder to compute group-specific scaling factors $\alpha$ (for learning rate) and $\beta$ (for weight decay). Unlike prior works that rely on hand-crafted per-layer heuristics, our method learns to modulate online from the optimization state, enabling genuinely adaptive per-group updates.

We further equip MetaAdamW with a meta-learning framework that periodically updates the attention module using a novel composite objective. This objective comprises three complementary terms: (i) gradient alignment between the original and a hypothetical one-step updated model, (ii) validation loss decrease, and (iii) generalization gap (training-validation loss difference). To automatically balance these terms, we adopt homoscedastic uncertainty weighting [3] (HUW). Crucially, we extend HUW by injecting task-specific priorities that directly scale the regularization terms ($\log \sigma$). This priority-injected HUW – a novel contribution – allows users to encode domain-specific importance while retaining automatic balancing. We validate its effectiveness through ablation studies.

We evaluate MetaAdamW on five representative tasks spanning sequence modeling, vision, and language:

- Time series forecasting (ETTh1) using a Transformer encoder,
- Language modeling (WikiText-2) using a MiniGPT model,
- Machine translation (Multi30k De→En) using a Transformer seq2seq,
- Image classification (CIFAR-10) using ResNet-18,
- Sentiment analysis (IMDB) using a bidirectional LSTM.

Compared to standard AdamW, MetaAdamW achieves:

- For Transformer-based tasks with fewer training epochs (ETTh1, WikiText-2): lower validation loss / perplexity (improvements of 4.26% and 4.12%) while reducing total training time by 7.20% and 17.11%, respectively.
- For the Transformer-based Multi30k translation task (more epochs): 2.99% lower perplexity with 27.35% longer training time, successfully mitigating premature early stopping.
- For non-Transformer architectures (ResNet-18, LSTM): accuracy gains of 1.18% and 11.08%, respectively, with increased training time (27.58% and 172.53%), again avoiding too-early stopping.

These results demonstrate that MetaAdamW not only improves generalization but also offers flexible trade-offs between performance and training cost depending on task characteristics.

Ablation studies further dissect the contributions of feature versions, grouping strategies, meta-objectives, and the proposed priority-injected uncertainty weighting. Our results confirm that the attention-based modulation and meta-learning are effective and generalizable across different architectures and tasks.

## 2 Related Work

### 2.1 Adaptive Optimization Algorithms

The Adam optimizer [4] combines momentum and RMSProp, providing adaptive learning rates per parameter. AdamW [6] decouples weight decay from the adaptive updates, improving generalization. Despite their success, these methods treat all parameters uniformly. Several works have attempted to introduce per-layer or per-parameter hyperparameters. For instance, HyperAdam [8] employs a recurrent meta-network to generate adaptive decay rates and

combination weights, requiring an additional meta-optimization loop. Meta-SGD [5] learns a parameter-wise learning rate vector and initialization via nested gradient-based meta-learning, but does not capture interactions among parameter groups. In contrast, MetaAdamW leverages self-attention to capture dependencies among parameter groups and computes modulation factors in a single forward pass, without introducing a separate meta-network.

### 2.2 Learning to Optimize with Recurrent Networks

Transformers and self-attention [7] have revolutionized sequence modeling. Recent works have explored learning to optimize using recurrent neural networks (e.g., LSTMs) [1] or hypernetwork-based adaptive optimizers [8]. However, these approaches either rely on hand-designed update rules (HyperAdam) or require a separate meta-optimizer with high computational cost. In contrast, MetaAdamW integrates a lightweight Transformer encoder directly into the optimizer to compute per-group modulation factors, leveraging self-attention to capture dependencies among parameter groups while incurring minimal overhead.

### 2.3 Meta-Learning for Optimization

Meta-learning aims to learn learning algorithms themselves. In the context of optimization, MAML [2] learns a model initialization that enables fast adaptation with few gradient steps. Another line of work, such as the LSTM-based optimizer [1] (often termed "learning to learn by gradient descent") and Meta-SGD [5], trains a meta-learner to directly produce parameter updates. Our meta-update scheme is inspired by these approaches but is tailored to the attention module within the optimizer. We adopt a bilevel optimization formulation where the inner loop is a single MetaAdamW step, and the outer loop minimizes a combined objective computed on separate mini-batches.

### 2.4 HUW and Priority Injection

Multi-task learning often requires balancing multiple loss terms. Kendall et al. [3] proposed using homoscedastic uncertainty (learnable task variances) to automatically weight losses. For a regression task, the log-likelihood is $-\frac{1}{2\sigma^2}|\mathbf{y}-\hat{\mathbf{y}}|^2 - \log\sigma$; for classification, a scaled softmax is used. The learnable $\sigma$ (or log-variance) automatically balances the losses. In this work, we extend this formulation by introducing *task-specific priorities* $p_i$ that scale the regularization term (but not the data term). The resulting loss for a regression task becomes $\sum_i \left(\frac{1}{2\sigma_i^2}\mathscr{L}_i + p_i \log\sigma_i\right)$, and for a classification task becomes $\sum_i \left(\frac{1}{\sigma_i^2}\mathscr{L}_i + p_i \log\sigma_i^2\right)$, following the approximation in [3] and consistent with our implementation.

## 3 Methodology

### 3.1 Problem Formulation

Consider a deep neural network with parameters $\theta$. Standard AdamW maintains first and second moments for each parameter. Let the parameter set be partitioned into $G$ groups $\{\mathscr{P}_g\}_{g=1}^{G}$ according to a grouping strategy (Sec. 3.2). For each group $g$, we aim to compute two modulation factors: $\alpha_g$ (scaling the learning rate) and $\beta_g$ (scaling the weight decay). The update rule for a parameter $p \in \mathscr{P}_g$ becomes:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla L(\theta_{t-1}), \tag{1}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla L)^2, \tag{2}$$

$$\hat{m}_t = m_t/(1-\beta_1^t), \quad \hat{v}_t = v_t/(1-\beta_2^t), \tag{3}$$

$$\theta_t = \theta_{t-1} - \eta\,\alpha_g\left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\varepsilon} + \beta_g\lambda\theta_{t-1}\right), \tag{4}$$

where $\eta$ is the global learning rate, $\lambda$ the weight decay, and $\varepsilon$ a small constant.

The core challenge is to compute $\alpha_g$ and $\beta_g$ dynamically based on the current optimization state.

### 3.2 Group Construction

We implement a fine-grained grouping strategy that categorizes parameters by layer type (embedding, attention, feed-forward, layer norm, other), depth bucket (shallow, middle, deep), and bias indicator. For the attention module, this grouping method represents a manageable scale. For models where internal structure is unavailable, we fall back to PyTorch's native parameter groups.

### 3.3 Feature Extraction

For each group $\mathscr{P}_g$, we extract a feature vector $\mathbf{f}_g \in \mathbb{R}^D$ that summarizes the optimization dynamics. We propose three core feature versions:

- **Basic**: 4 statistics (mean of gradient norms, momentum norms, parameter norms, cosine similarity between gradient and momentum) plus optional time step and optional second-order momentum norm.
- **Basic+**: adds standard deviations of the four basic statistics.
- **Enhanced**: 9 statistics (mean and variance of gradient norms, mean and variance of momentum norms, gradient sparsity, momentum sparsity, log parameter count, bias ratio, normalized layer depth) plus time step and a learnable group embedding.

Additionally, we provide normalized variants that perform cross-group standardization on the statistical columns.

### 3.4 Attention-Based Modulation

Let $\mathbf{F} = [\mathbf{f}_1;\ldots;\mathbf{f}_G] \in \mathbb{R}^{G\times D}$ be the feature matrix. We apply a Transformer encoder with $L$ layers, $H$ heads, and hidden dimension $D_{\text{ff}}$:

$$\mathbf{Z} = \text{TransformerEncoder}(\mathbf{F}), \quad \mathbf{Z} \in \mathbb{R}^{G\times D}. \tag{5}$$

A linear projection outputs four raw values per group:

$$[\alpha_g^{\text{raw}}, \beta_g^{\text{raw}}, \lambda_{1,g}^{\text{raw}}, \lambda_{2,g}^{\text{raw}}]^\top = \mathbf{W}_{\text{out}}\mathbf{z}_g + \mathbf{b}_{\text{out}}. \tag{6}$$

The final modulation factors are:

$$\alpha_g = 1 + \text{range}_\alpha \cdot (\sigma(\alpha_g^{\text{raw}}) - 0.5), \tag{7}$$

$$\beta_g = 1 + \text{range}_\beta \cdot (\sigma(\beta_g^{\text{raw}}) - 0.5), \tag{8}$$

$$\lambda_{1,g} = \sigma(\lambda_{1,g}^{\text{raw}}), \quad \lambda_{2,g} = \sigma(\lambda_{2,g}^{\text{raw}}), \tag{9}$$

where $\sigma$ is the sigmoid function. $\lambda_{1,g}$ and $\lambda_{2,g}$ are used in the meta-learning objective (Sec. 3.5).

### 3.5 Meta-Learning Objectives

We periodically update the attention module (every $K_{\text{meta}}$ steps) to minimize a combined objective that promotes desirable optimization properties. The meta-update uses two separate mini-batches $\mathscr{B}_1$ and $\mathscr{B}_2$ (drawn from the training set) and a validation batch $\mathscr{B}_{\text{val}}$.

Let $\theta$ be the model parameters, and let $\theta'$ denote the parameters after one MetaAdamW step using the current attention module. The meta-loss consists of three terms:

1. **Gradient alignment** ($\mathscr{L}_{\text{grad}}$): Encourages the gradient of the updated model on $\mathscr{B}_2$ to align with the original gradient on $\mathscr{B}_1$:

$$\mathscr{L}_{\text{grad}} = \frac{1}{G}\sum_{g=1}^{G}\left(\lambda_{1,g}\|\mathbf{g}'_g\|^2 - \lambda_{2,g}\cos(\mathbf{g}_g, \mathbf{g}'_g)\right), \tag{10}$$

where $\mathbf{g}_g$ is the aggregated gradient of group $g$ on the first mini-batch $\mathscr{B}_1$, and $\mathbf{g}'_g$ is the aggregated gradient of the same group on $\mathscr{B}_2$ *after* the hypothetical one-step update.

2. **Loss decrease** ($\mathscr{L}_{\text{loss}}$): Measures the reduction in validation loss:

$$\mathscr{L}_{\text{loss}} = L_{\text{val}}(\theta') - L_{\text{val}}(\theta). \tag{11}$$

3. **Generalization gap** ($\mathscr{L}_{\text{gap}}$): Penalizes the difference between training and validation losses:

$$\mathscr{L}_{\text{gap}} = |L_{\text{train}}(\theta, \mathscr{B}_1) - L_{\text{val}}(\theta)|. \tag{12}$$

Meta-learning objectives can be selected from four options: Gradient alignment, Loss decrease, Generalization gap, and Combined.

When the meta-learning objective is Combined, instead of fixed weights, we learn the weights adaptively using homoscedastic uncertainty [3]. For each term $i$, we introduce a learnable log-variance $s_i = \log\sigma_i^2$. Since all three meta-loss terms ($\mathscr{L}$grad, $\mathscr{L}$loss, $\mathscr{L}$gap) are scalar regression objectives, we adopt the regression formulation. The meta-loss is defined as:

$$\mathscr{L}\text{meta} = \sum_{i=1}^{3}\left(\frac{1}{2\sigma_i^2}\mathscr{L}_i + p_i \cdot \frac{1}{2}\log\sigma_i^2\right), \tag{13}$$

where $p_i$ is a user-defined priority for task $i$. (For completeness, a classification task would use $\frac{1}{\sigma_i^2}\mathscr{L}_i + p_i \log\sigma_i^2$ instead, following our implementation.) To the best of our knowledge, this priority injection into the HUW has not been previously proposed; it offers a flexible way to incorporate domain knowledge while preserving automatic balancing.

### 3.6 Algorithm Summary

The optimizer maintains standard AdamW states and additionally updates the attention module every $K_{\text{meta}}$ steps via the meta-objective. In practice, the meta-update is not embedded inside the main `step()` function; rather, the user is responsible for invoking `update_attention()` at the desired frequency after the warmup phase. Algorithm 1 outlines the logical flow of a single training iteration, assuming the meta-update is triggered externally when the condition holds.

Algorithm 1 outlines the overall procedure. The optimizer maintains standard AdamW states and additionally updates the attention module every $K_{\text{meta}}$ steps via the meta-objective.

The key steps of the meta-update algorithm (Appendix A) are:

- **Save original parameters**: First, the original model parameters are saved;
- **Compute original gradients**: then, gradients are computed on the first mini-batch $\mathscr{B}_1$ using the current model.
- **Compute modulation factors**: Modulation factors $(\alpha_{\text{g}}, \beta_{\text{g}}, \lambda_{1,\text{g}}, \lambda_{2,\text{g}})$ are obtained from the attention module.
- **Construct hypothetical parameters**: Hypothetical parameters $\theta'$ are constructed by applying one MetaAdamW step.
- **Compute gradients on hypothetical parameters**: Gradients $\mathbf{g}'$ are computed on $\theta'$ using the second mini-batch $\mathscr{B}_2$.

**Algorithm 1** MetaAdamW Update (one step)

**Input:** Model $\theta$, learning rate $\eta$, weight decay $\lambda$, meta-update frequency $K_{\text{meta}}$, global step $t$.
**Output:** Updated $\theta$, attention module $\Phi$.

```
                                   ▷ Standard AdamW states: m, v
Compute groups {𝒫_g} and extract features {f_g}.
Compute modulation factors α_g, β_g via Φ (attention encoder + output projection).
for each parameter p in group g do
    Update m_t, v_t using current gradient.
    Apply update: p ← p − η α_g (m̂_t / (√v̂_t + ε) + β_g λ p).
end for
if t mod K_meta = 0 and t ≥ warmup_steps then
    Note: In practice, the meta-update is not executed automatically inside step(); instead, the training loop must explicitly call update_attention() every K_meta steps after the warmup period. The pseudocode below assumes this external invocation.
    Perform meta-update of Φ using two training batches and a validation batch (Algorithm 2).
end if
t ← t + 1
```

- **Compute meta-loss**: A meta-loss $\mathscr{L}_{\text{meta}}$ (combining gradient alignment, loss decrease, and generalization gap, weighted by priority-injected homoscedastic uncertainty) is evaluated,
- **Backpropagate to update attention module**: and backpropagated through the attention module to update its parameters.
- **Restore original parameters**: Finally, the original model parameters are restored.

## 4 Experiments

### 4.1 Experimental Setup

#### 4.1.1 Tasks and Datasets

To validate the effectiveness of MetaAdamW and to demonstrate its generality, we conduct experiments on five fundamentally different tasks:

- **Time Series Forecasting (ETT)**: ETTh1 dataset (hourly, 7 features). Input length 96, predict next 1 step (oil temperature). Model: Transformer encoder with $d_{\text{model}} = 128$, 2 layers, 4 heads.
- **Language Modeling (WikiText-2)**: Word-level language modeling with block size 128, vocabulary size ≈10k. Model: MiniGPT with $d_{\text{model}} = 128$, 2 layers, 4 heads.
- **Machine Translation (Multi30k)**: German→English translation, sequence length 64. Model: Transformer seq2seq with $d_{\text{model}} = 256$, 4 encoder/decoder layers, 8 heads.
- **Image Classification (CIFAR-10)**: 10-class classification. Model: ResNet-18 (modified for 32×32 inputs).
- **Sentiment Analysis (IMDB)**: Binary sentiment classification, sequence length 256. Model: Bidirectional LSTM with embedding dimension 32, hidden size 64, 2 layers.

#### 4.1.2 Baseline and Hyperparameters

We compare MetaAdamW against standard AdamW (decoupled weight decay). For fairness, both optimizers use the same base learning rate $5e-4$, weight decay $1e-2$, and betas $(0.9, 0.999)$. MetaAdamW hyperparameters that are common across tasks are

listed in Table 1. Task-specific optimal hyperparameters (e.g., meta-update frequency, attention depth, feature version, HUW priorities) are provided in Appendix B.

Table 1: MetaAdamW Common Hyperparameters

| Parameter | Value |
|---|---|
| Global learning rate $\eta$ | $5 \times 10^{-4}$ |
| Weight decay $\lambda$ | $1 \times 10^{-2}$ |
| AdamW betas | $(0.9, 0.999)$ |
| $\alpha$ range | 1.0 |
| $\beta$ range | 1.0 |
| Warmup epochs (before meta) | 1 |
| Group strategy | fine-grained |
| Feature version | basic (or basic_plus for LSTM) |
| Use $v$ norms | True |
| Include time step | True |
| Meta objective | combined |

#### 4.1.3 Evaluation Metrics

We report:

- **Validation loss** - ETT: MSE; LM: cross-entropy; MT: cross-entropy; CIFAR/IMDB: cross-entropy.
- **Task-specific metric** - ETT: MSE; LM: perplexity; MT: perplexity; CIFAR/IMDB: accuracy.
- **Total training time** - (seconds) and relative change.

Early stopping with patience 2 is applied for both optimizers.

### 4.2 Results

#### 4.2.1 Performance Comparison

Table 2 reports the best validation metrics and total training time for AdamW and MetaAdamW on all five tasks. MetaAdamW consistently outperforms the baseline on every task, with improvements ranging from modest (1.18% accuracy on CIFAR-10) to substantial (11.08% accuracy on IMDB). Notably, for two Transformer-based tasks (ETTh1, WikiText-2), MetaAdamW achieves better performance (4.26% and 4.12%) while also reducing total training time (by 7.20% and 17.11%, respectively), because it reaches a better optimum earlier and triggers early stopping sooner. For the more complex Multi30k translation task, MetaAdamW avoids premature early stopping (training for more epochs) and yields a 2.99% perplexity reduction at a 27.35% time cost. For non-Transformer architectures (ResNet-18, LSTM), MetaAdamW also trains longer (27.58% and 172.53%) but delivers clear accuracy gains (1.18% and 11.08%).

Figure 1, 2, 3, 4 and 5 show the validation curves for each task. MetaAdamW may converge faster, or ultimately perform better, or help avoid premature stopping and achieve better convergence, depending on the task.

#### 4.2.2 Computational Overhead

The additional cost of MetaAdamW comes from feature extraction, attention forward/backward, and meta-updates. As shown in Table 2, the total training time change varies: it decreases for ETTh1 and WikiText-2 (due to earlier stopping) but increases for the other three tasks. The largest relative increase occurs on IMDB (172.5%), because the LSTM model has many parameter groups and the meta-update frequency was set high (every 39 steps). Nevertheless, the performance gains justify the extra cost in those scenarios.

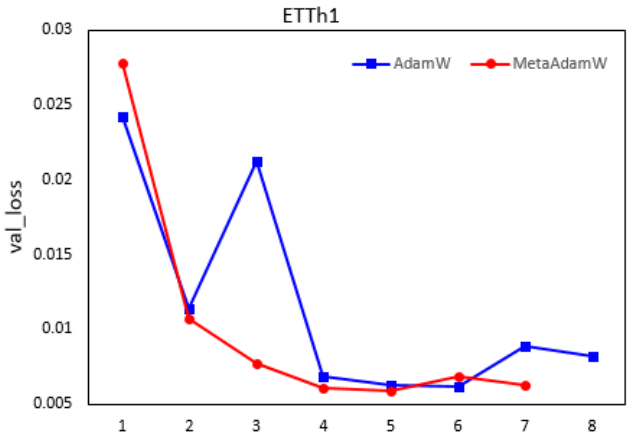


Figure 1: ETTh1 Validation loss over epochs.

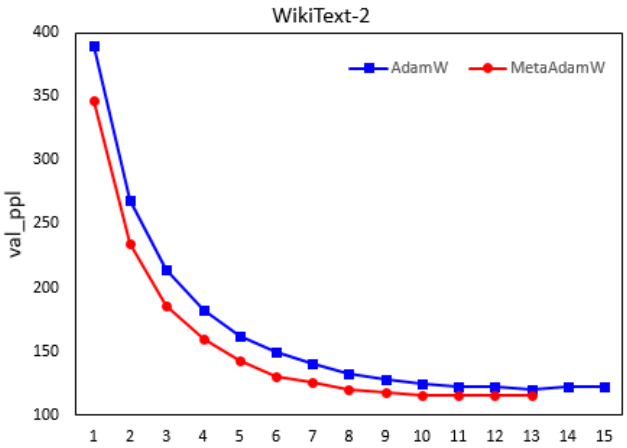


Figure 2: WikiText-2 Validation perplexity over epochs.

### 4.3 Ablation Studies

To systematically assess the contribution of each design component, we conducted extensive ablations across all five tasks. Table 3 reports representative results on five tasks. All hyperparameters except the ablated one follow the task-optimal settings (Appendix B).

Grouping strategy. Fine-grained grouping (by layer type, depth, and bias) consistently outperforms PyTorch's native parameter groups.

Feature version. The simple `basic` feature set (mean of gradient / momentum / parameter norms and cosine similarity, plus time step and second-moment norm) achieves near-optimal results. Adding standard deviations (`basic_plus`) hurts performance. The more complex `enhanced` features perform even worse. Thus, `basic` is sufficient for most tasks.

Meta-learning objective. Removing the attention modulation (i.e., standard AdamW) causes substantial degradation. The `combined` objective outperforms any single component.

HUW priorities. Priority-injected homoscedastic uncertainty weighting (HUW) improves over fixed equal weights. This validates the benefit of task-specific priority scaling.

Feature gating. Learnable feature gating still requires further research and exploration.

Table 2: Main Results: Best Validation Metrics and Total Training Time

| Task | AdamW | | MetaAdamW | | Improvement | Time Overhead |
|---|---|---|---|---|---|---|
| | Metric | Value | Metric | Value | | |
| ETTh1 (forecast) | MSE | 0.006147 | MSE | 0.005885 | +4.26% | -7.20% |
| WikiText-2 (LM) | PPL | 120.47 | PPL | 115.51 | +4.12% | -17.11% |
| Multi30k (MT) | PPL | 2.0297 | PPL | 1.9690 | +2.99% | +27.35% |
| CIFAR-10 (classif.) | Acc | 87.02% | Acc | 88.05% | +1.18% | +27.58% |
| IMDB (sentiment) | Acc | 74.53% | Acc | 82.79% | +11.08% | +172.53% |

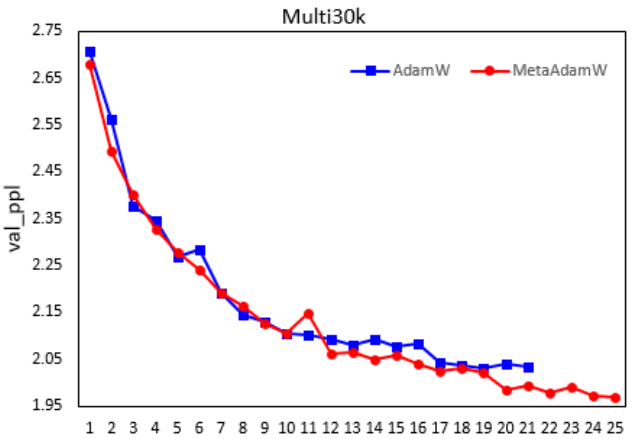


Figure 3: Multi30k Validation perplexity over epochs.

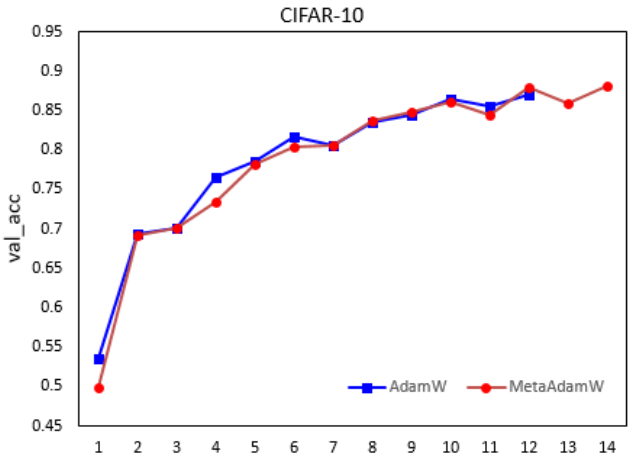


Figure 4: CIFAR-10 Validation accuracy over epochs.

Meta-update frequency. An excessively high meta-update frequency significantly increases computational time and does not necessarily yield substantial performance improvements; to a large extent, this depends on the specific task.

Attention module size. More complex tasks benefit from greater expressivity.

In summary, the ablation study confirms that the substantial performance of self-attentive MetaAdamW optimizer stems from the synergy of fine-grained grouping, the combined meta-objective, and the priority-injected HUW balancing.

### 4.4 Discussion

The results demonstrate that MetaAdamW effectively learns to modulate per-group learning rates and weight decay. The meta-learning component adapts the attention module to the specific task and training dynamics, leading to better generalization. The computational overhead is task-dependent: for tasks where early stopping is triggered earlier (ETTh1, WikiText-2), total time actually decreases; for others, the extra time is compensated by improved accuracy or perplexity.

A notable observation is that the optimal hyperparameters of MetaAdamW vary substantially across tasks. For instance, the LSTM sentiment task required a higher attention head count (11 vs. 6) and the `basic_plus` feature version, while the Transformer-based tasks worked well with the simpler `basic` features. The meta-update frequency also differed: 217 steps for ETTh1, 123 for WikiText-2, 454 for Multi30k, 190 for CIFAR-10, and 39 for IMDB. This diversity underscores the flexibility of MetaAdamW and suggests that practitioners may need to tune a few key parameters for their specific domain. We provide the full set of optimal configurations in Appendix B to facilitate reproducibility.

### 4.5 Memory and Computation Overhead

We analyze the peak memory and computational cost of MetaAdamW compared to the baseline AdamW, as this is critical for practical deployment.

**Peak memory footprint.** Standard AdamW stores model parameters ($P$), gradients ($P$), first-moment ($P$) and second-moment ($P$) buffers, plus forward activations and the backward computation graph. Its peak memory is approximately $3P$–$4P + \text{activations} + \text{graph}$.

MetaAdamW introduces an additional peak during meta-update steps (every $K_{\text{meta}}$ iterations). In such a step, the following coexist:

- Original model parameters, gradients (‘grads1‘), and momentum buffers (total $\sim 4P$)
- Temporary parameters ‘temp_params‘ (a full copy of the model, size $P$)
- Second-order gradients ‘grads_temp‘ (size $P$)
- Two sets of forward activations (for batches 1 and 2)
- A second backward computation graph

Consequently, the **peak memory during a meta-update** can reach roughly $5P$–$6P + 2 \times \text{activations} + \text{graph}_2$, i.e., **1.5–2**$\times$ the peak of AdamW. During regular (non-meta) steps, MetaAdamW's peak memory is nearly identical to AdamW because no temporary parameters or second backward pass are created.

**Computational cost.** In a regular step, feature extraction adds about 0.5-1$\times$ the FLOPs of the baseline AdamW update, while the attention module contributes negligible overhead. Meta-updates require an extra full forward-backward pass ($\sim 1\times$ model computation) every $K_{\text{meta}}$ steps. With $K_{\text{meta}} = 100$, the average extra computation is roughly 1% of model computation plus the per-step feature

Table 3: Ablation study: validation performance changes.

| Configuration | ETTh1 (Loss) | | WikiText-2 (PPL) | | Multi30k (PPL) | | CIFAR-10 (Acc) | | IMDB (Acc) | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Value | Δ% | Value | Δ% | Value | Δ% | Value | Δ% | Value | Δ% |
| MetaAdamW (baseline) | 0.005885 | – | 115.51 | – | 1.9690 | – | 0.8805 | – | 0.8279 | – |
| AdamW | 0.006147 | -4.45 | 120.47 | -4.29 | 2.0297 | -3.08 | 0.8702 | -1.17 | 0.7453 | -9.98 |
| **Grouping strategy** | | | | | | | | | | |
| Original | – | – | 116.13 | -0.54 | – | – | – | – | – | – |
| Fine-grained | baseline | – | baseline | – | baseline | – | baseline | – | baseline | – |
| **Feature version** | | | | | | | | | | |
| Basic | baseline | – | baseline | – | baseline | – | baseline | – | 0.7938 | -4.12 |
| Basic plus | – | – | 117.78 | -1.97 | 2.6180 | -32.96 | – | – | baseline | – |
| Enhanced | – | – | 121.02 | -4.77 | – | – | – | – | – | – |
| **Meta-learning objective** | | | | | | | | | | |
| Gradient | – | – | 116.13 | -0.54 | – | – | 0.8304 | -5.69 | – | – |
| Loss decrease | – | – | 116.70 | -1.03 | – | – | – | – | – | – |
| Generalization gap | – | – | 116.47 | -0.83 | – | – | – | – | – | – |
| Combined | baseline | – | baseline | – | baseline | – | baseline | – | baseline | – |
| **HUW priorities** | | | | | | | | | | |
| No HUW | – | – | 116.13 | -0.54 | – | – | – | – | – | – |
| [1.0, 1.0, 1.0] | baseline | – | baseline | – | 2.1205 | -7.69 | 0.8602 | -2.31 | – | – |
| Not [1.0, 1.0, 1.0] | – | – | 115.57 | -0.05 | baseline | – | baseline | – | baseline | – |
| **Feature gating** | | | | | | | | | | |
| Enable | baseline | – | 116.17 | -0.57 | 2.6180 | -32.96 | 0.8602 | -2.31 | baseline | – |
| Disable | 0.005932 | -0.80 | baseline | – | baseline | – | baseline | – | 0.7938 | -4.12 |
| **Meta-update frequency** | | | | | | | | | | |
| Once per epoch | baseline | – | baseline | – | baseline | – | 0.8776 | -0.33 | 0.6767 | -18.26 |
| Approx. twice per epoch | – | – | 116.13 | -0.54 | – | – | baseline | – | – | – |
| Approx. 10 times per epoch | – | – | 116.52 | -0.87 | 2.0338 | -3.29 | – | – | baseline | – |
| **Attention Module** | | | | | | | | | | |
| $L_{\text{attn}}$=8, $D_{\text{attn}}$=16 | baseline | – | baseline | – | 2.0338 | -3.29 | 0.8579 | -2.57 | – | – |
| Not ($L_{\text{attn}}$=8, $D_{\text{attn}}$=16) | – | – | 116.32 | -0.70 | baseline | – | baseline | – | baseline | – |

*Notes: (1) Δ% indicates relative change from baseline; negative Δ% means worse performance. (2) "–" means no data or baseline.*

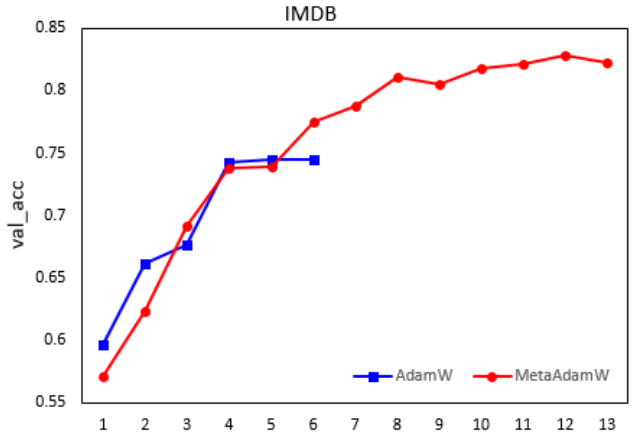


Figure 5: IMDB Validation accuracy over epochs.

extraction cost. In practice, end-to-end training time **increases by 10–30%**.

### 4.6 Future Work

While MetaAdamW is validated on lightweight models, scaling it to billion-parameter Transformers and more complex tasks remains an important direction for further investigation. The current implementation incorporates a feature gating mechanism, which applies a learnable gate to each feature — mediated by a Sigmoid activation function — and is subsequently complemented by L1 sparsity regularization. Its effectiveness has been demonstrated in the current experiments; we will conduct further in-depth research into the feature gating mechanism.

## 5 Conclusion

We have introduced MetaAdamW, a new optimizer that integrates self-attention and meta-learning into the AdamW framework. It solves the problem of uniform hyperparameter treatment by extracting statistical features from parameter groups and using a Transformer encoder to compute adaptive, per-group learning rates and weight decay. A novel meta-learning objective – combining gradient alignment, loss decrease, and generalization gap – balances these terms via priority-injected homoscedastic uncertainty weighting, a mechanism we propose and validate. Our experiments across five diverse tasks (ETTh1, WikiText-2, Multi30k, CIFAR-10, IMDB) show that MetaAdamW consistently outperforms standard AdamW. Depending on the task, it either reduces total training time (by up to 17.11%) or achieves better performance with modest overhead (e.g., 2.99% lower perplexity on Multi30k, 11.08% higher accuracy on IMDB) while mitigating premature early stopping. Ablation studies confirm the necessity of each design component.

## References


[1] M. Andrychowicz, M. Denil, S. Gomez, M. W. Hoffman, D. Pfau, T. Schaul, B. Shillingford, and N. de Freitas. Learning to learn by gradient descent by gradient descent, 2016.

[2] C. Finn, P. Abbeel, and S. Levine. Model-agnostic meta-learning for fast adaptation of deep networks, 2017.

[3] A. Kendall, Y. Gal, and R. Cipolla. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics, 2018.
[4] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization, 2017.
[5] Z. Li, F. Zhou, F. Chen, and H. Li. Meta-sgd: Learning to learn quickly for few-shot learning, 2017.
[6] I. Loshchilov and F. Hutter. Decoupled weight decay regularization, 2019.
[7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need, 2023.
[8] S. Wang, J. Sun, and Z. Xu. Hyperadam: A learnable task-adaptive adam for network training, 2018.

## A Meta-Update Algorithm

Algorithm 2 computes the hypothetical parameters $\theta'$ using current $\Phi$, evaluates $\mathscr{L}_{\text{meta}}$, and backpropagates through the attention module.

## B Task-Specific Hyperparameters

Table 4 reports the optimal hyperparameter configuration for each task, as discovered during our ablation sweeps. These values can serve as a starting point for future applications.

## C Detailed Training Time Breakdown

Table 5 shows the average epoch time (seconds) and total training time for AdamW vs. MetaAdamW.

## D Generative AI Usage Disclosure

The authors used generative AI tools (DeepSeek) for language refinement and code adaptation. All scientific content, experimental design, and conclusions are the sole responsibility of the authors.

## E Repository

https://github.com/qq150078158-lab/MetaAdamW

**Algorithm 2** Meta-Update of Attention Module

**Input:** Model $\theta$ (current parameters), attention module $\Phi$ (including group embeddings), two training batches $\mathscr{B}_1, \mathscr{B}_2$, validation batch $\mathscr{B}_{\text{val}}$ (if needed), loss function $\mathscr{L}_{\text{task}}$, meta-learning rate $\eta_{\text{meta}}$, current optimizer states $\{m, v\}$.

**Output:** Updated attention module $\Phi$.

▷ Step 1: Save original parameters

**for** each parameter $p$ in $\theta$ **do**

  $p_{\text{orig}} \leftarrow p.\text{data.clone}()$

**end for**

▷ Step 2: Compute original gradients $\mathbf{g}$ on $\mathscr{B}_1$

$\mathscr{L}_1 \leftarrow \mathscr{L}_{\text{task}}(\theta, \mathscr{B}_1)$

$\mathbf{g} \leftarrow \nabla_\theta \mathscr{L}_1$ ▷ gradients w.r.t. all parameters

▷ Step 3: Compute modulation factors using current $\Phi$

Extract group features $\{\mathbf{f}_g\}$ from current states and time step.

$(\alpha_g, \beta_g, \lambda_{1,g}, \lambda_{2,g}) \leftarrow \Phi(\{\mathbf{f}_g\})$ ▷ as in Algorithm 1

▷ Step 4: Build hypothetical parameters $\theta'$ after one AdamW step

**for** each parameter group $g$ **do**

  **for** each parameter $p \in \mathscr{P}_g$ **do**

    Compute $m', v'$ using standard AdamW update (Eq. 1–3) with $\alpha_g, \beta_g$.

    $\theta'_p \leftarrow p - \eta \alpha_g \left( \frac{\hat{m}'}{\sqrt{\hat{v}'} + \varepsilon} + \beta_g \lambda p \right)$

  **end for**

**end for**

▷ Step 5: Compute gradients $\mathbf{g}'$ on $\mathscr{B}_2$ using $\theta'$

$\mathscr{L}_2 \leftarrow \mathscr{L}_{\text{task}}(\theta', \mathscr{B}_2)$

$\mathbf{g}' \leftarrow \nabla_{\theta'} \mathscr{L}_2$

▷ Step 6: Compute meta-loss $\mathscr{L}_{\text{meta}}$

$\mathscr{L}_{\text{grad}} \leftarrow \frac{1}{G} \sum_{g=1}^{G} \left( \lambda_{1,g} \|\mathbf{g}'_g\|^2 - \lambda_{2,g} \cos(\mathbf{g}_g, \mathbf{g}'_g) \right)$

**if** meta objective includes loss decrease **then**

  $\mathscr{L}_{\text{loss}} \leftarrow \mathscr{L}_{\text{task}}(\theta', \mathscr{B}_{\text{val}}) - \mathscr{L}_{\text{task}}(\theta, \mathscr{B}_{\text{val}})$

**end if**

**if** meta objective includes generalization gap **then**

  $\mathscr{L}_{\text{gap}} \leftarrow |\mathscr{L}_{\text{task}}(\theta, \mathscr{B}_1) - \mathscr{L}_{\text{task}}(\theta, \mathscr{B}_{\text{val}})|$

**end if**

$\mathscr{L}_{\text{meta}} \leftarrow \text{CombinedObjective}(\mathscr{L}_{\text{grad}}, \mathscr{L}_{\text{loss}}, \mathscr{L}_{\text{gap}})$ ▷ weighted by HUW with priorities

▷ Step 7: Backpropagate through $\Phi$ and update

$\nabla_\Phi \mathscr{L}_{\text{meta}} \leftarrow \text{autograd}(\mathscr{L}_{\text{meta}})$

**for** each parameter $\phi$ in $\Phi$ (including group embeddings) **do**

  $\phi \leftarrow \phi - \eta_{\text{meta}} \nabla_\phi \mathscr{L}_{\text{meta}}$

**end for**

▷ Step 8: Restore original parameters

**for** each parameter $p$ in $\theta$ **do**

  $p.\text{data} \leftarrow p_{\text{orig}}$

**end for**

Table 4: Optimal MetaAdamW hyperparameters per task

| Task | $K_{\text{meta}}$ | $L$ (attn layers) | $D_{\text{hid}}$ (attn) | Heads | Feature version | Feature dim | Feature gating | HUW priorities |
|---|---|---|---|---|---|---|---|---|
| ETTh1 | 217 | 8 | 16 | 6 | basic | 6 | True | [1.0, 1.0, 1.0] |
| WikiText-2 | 123 | 8 | 16 | 6 | basic | 6 | False | [1.0, 1.0, 1.0] |
| Multi30k | 454 | 128 | 64 | 6 | basic | 6 | False | [2.0, 5.0, 1.0] |
| CIFAR-10 | 190 | 64 | 64 | 6 | basic | 6 | False | [5.0, 2.0, 1.0] |
| IMDB | 39 | 64 | 64 | 11 | basic_plus | 11 | True | [1.0, 3.0, 5.0] |

*Note: The HUW priorities correspond to the three meta-loss terms in the order: gradient alignment, loss decrease, generalization gap.*

Table 5: Average epoch time (seconds) and total training time

| Task | AdamW epoch | MetaAdamW epoch | AdamW total | MetaAdamW total | Overhead |
|---|---|---|---|---|---|
| ETTh1 | 93.39 | 99.04 | 747 | 693 | -7.20% |
| WikiText-2 | 373.58 | 357.32 | 5604 | 4645 | -17.11% |
| Multi30k | 895.25 | 957.67 | 18800 | 23942 | +27.35% |
| CIFAR-10 | 2410.36 | 2635.86 | 28924 | 36902 | +27.58% |
| IMDB | 283.35 | 356.40 | 1700 | 4633 | +172.53% |